LAB REPORT

CSE312: Database Management System Lab

Submitted To
Syed Eftasum Alam (SEA)
(Lecturer)
Department of CSE, Daffodil International University

Submitted By
Student ID**: 232-15-818**
Section**: 66_L2**
Student Name: **Md. Emon Hossain**

Submission Date: 10 April 2026

Experiment No: 04	Experiment Name: Implementation of Database Triggers, Procedures, and Views

1. Objective of the Experiment

The objective of this experiment is to gain practical experience in automating database tasks using **Triggers**, performing complex calculations with **Stored Procedures**, and simplifying data retrieval using **SQL Views**.

2. Experiment Details

Task 1:

Design a trigger named **grade_change_log** to log grade updates in an **updated_grade** table only when the grade actually changes.

Task 2:

Create a stored procedure **All_students_CGPA** to calculate the CGPA of all students based on the formula:

Σ(GPA × Credit) / Σ(Credit)

Task 3:

Create a SQL view **Avg_Result** to display the average CGPA per department, showing:

- dept_id
- dept_name
- avg(cgpa)

.

3. Software & Tools Used:

- Web Server Stack: XAMPP Control Panel v3.3.0
- Database Management System: MariaDB 10.4.x (MySQL-compatible)
- Database Server: MySQL Server (running on localhost via XAMPP)
- Interface Tool: phpMyAdmin 5.2.x

- Web Server: Apache 2.4.x
- Scripting Language: PHP 8.0.x
- Operating System: Windows 11

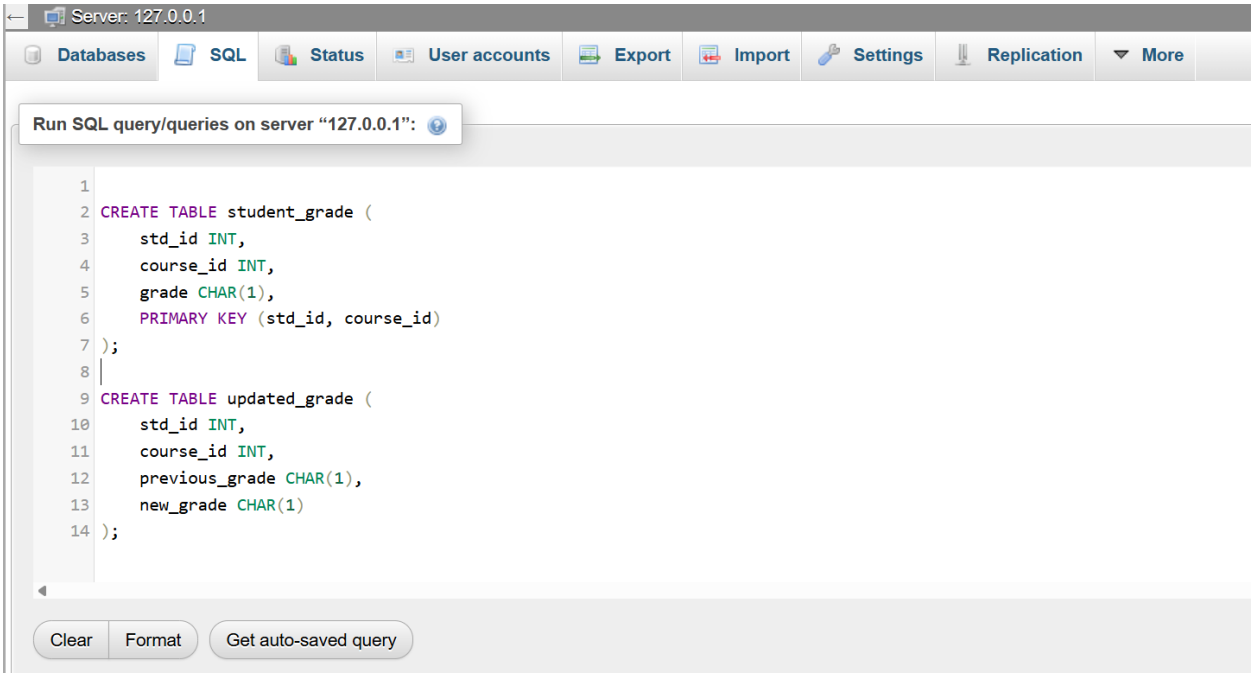
4. Methodology

1. Defined the schema for **student_grade**, **updated_grade**, **Department**, and **Students** tables.
2. Wrote and executed the **CREATE TRIGGER** syntax with an **IF** condition to check for grade changes.
3. Developed the **STORED PROCEDURE** using aggregate functions to compute weighted averages for CGPA.
4. Wrote a **CREATE VIEW** statement using a JOIN between **Department** and **Students** tables grouped by department.

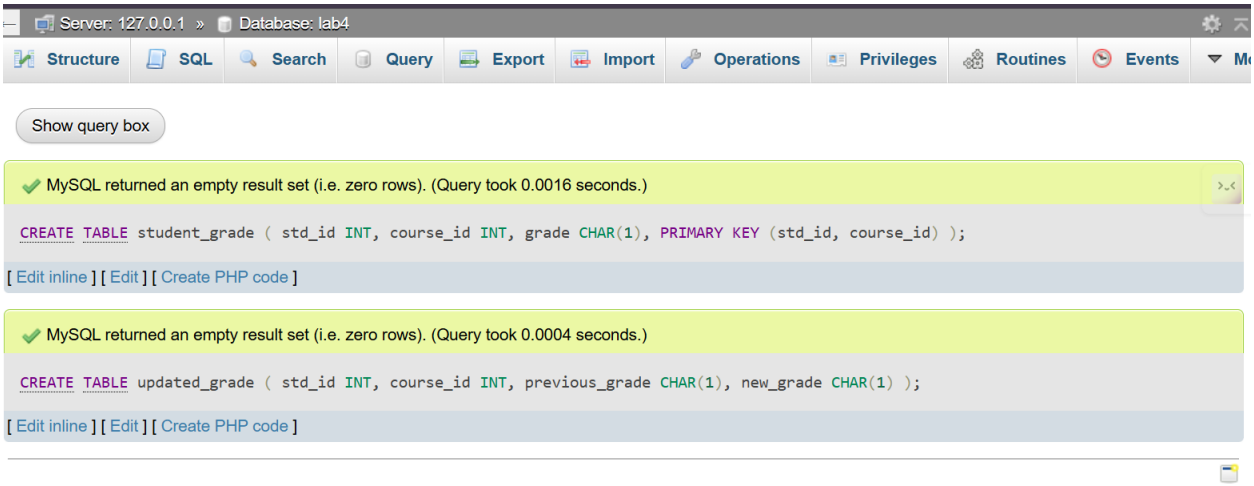
5. Experiment Solutions:

Q1. Design a trigger named grade_change_log to log grade updates in an updated_grade table only when the grade actually changes.

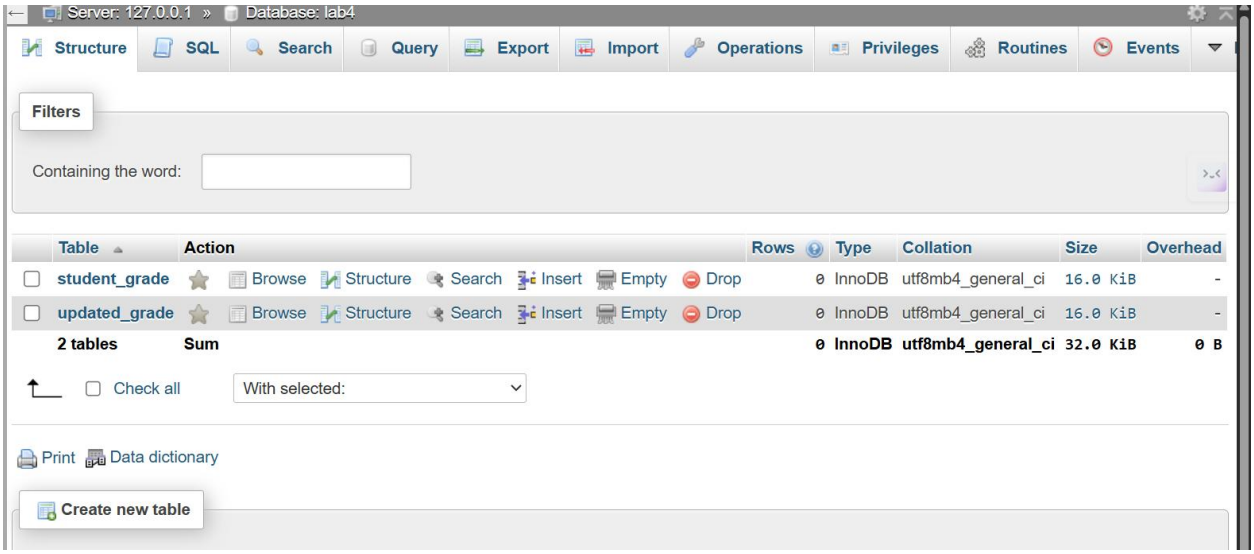
<u>Step-01: SQL Queries - Create Table</u>



Step-02: Success of Compilation



Step-03: Output

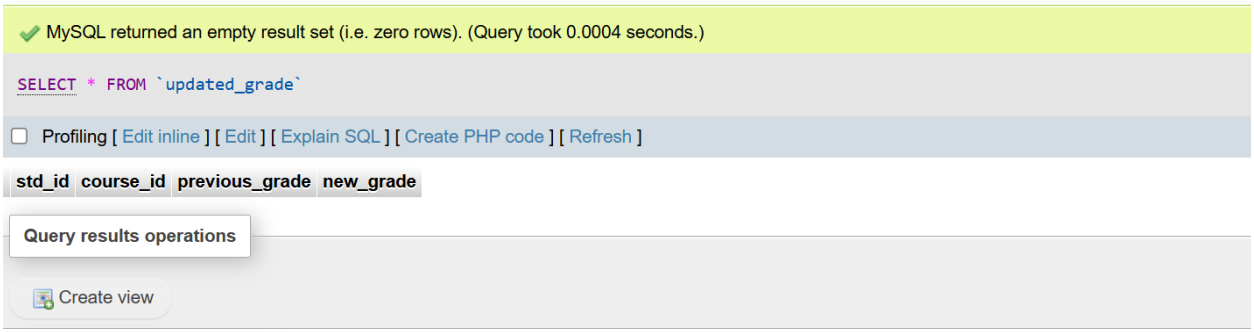


Step-04: Query 1: Trigger for Grade Change

```
DELIMITER $$
CREATE TRIGGER grade_change_log
AFTER UPDATE ON student_grade
FOR EACH ROW
BEGIN
    IF NEW.grade <> OLD.grade THEN
        INSERT INTO updated_grade (std_id, course_id, previous_grade, new_grade)
        VALUES (OLD.std_id, OLD.course_id, OLD.grade, NEW.grade);
    END IF;
END$$
DELIMITER ;
```

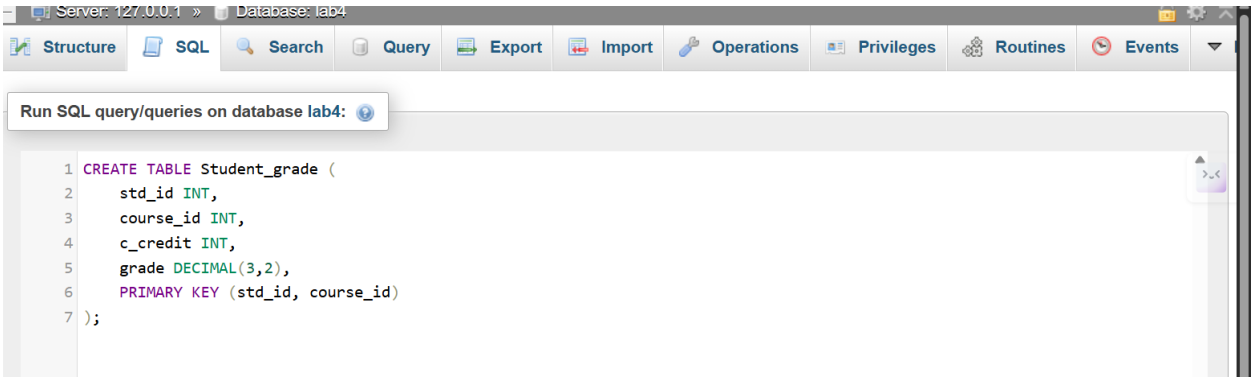
Step-05: Success of Compilation

Step-06: Output (Trigger for Grade Change)

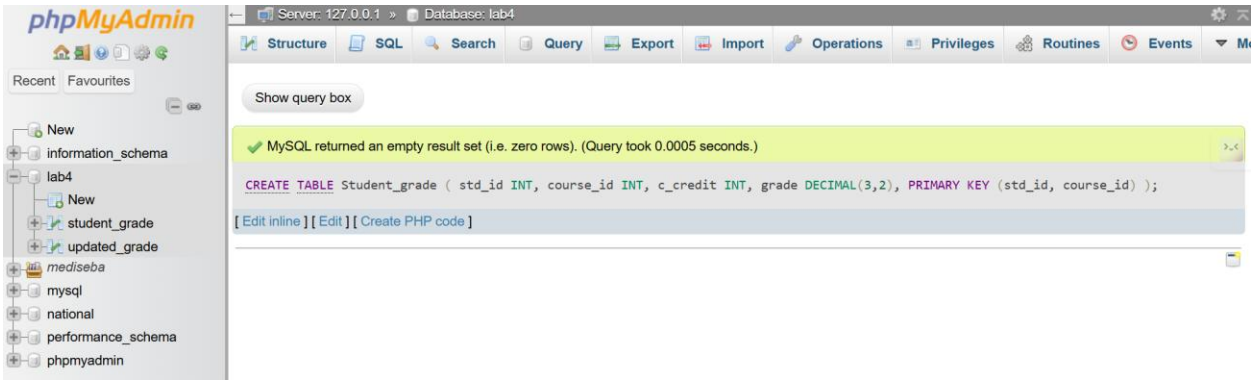


Q2. Create a stored procedure All_students_CGPA to calculate the CGPA of all students based on the formula:
Σ(GPA × Credit) / Σ(Credit)

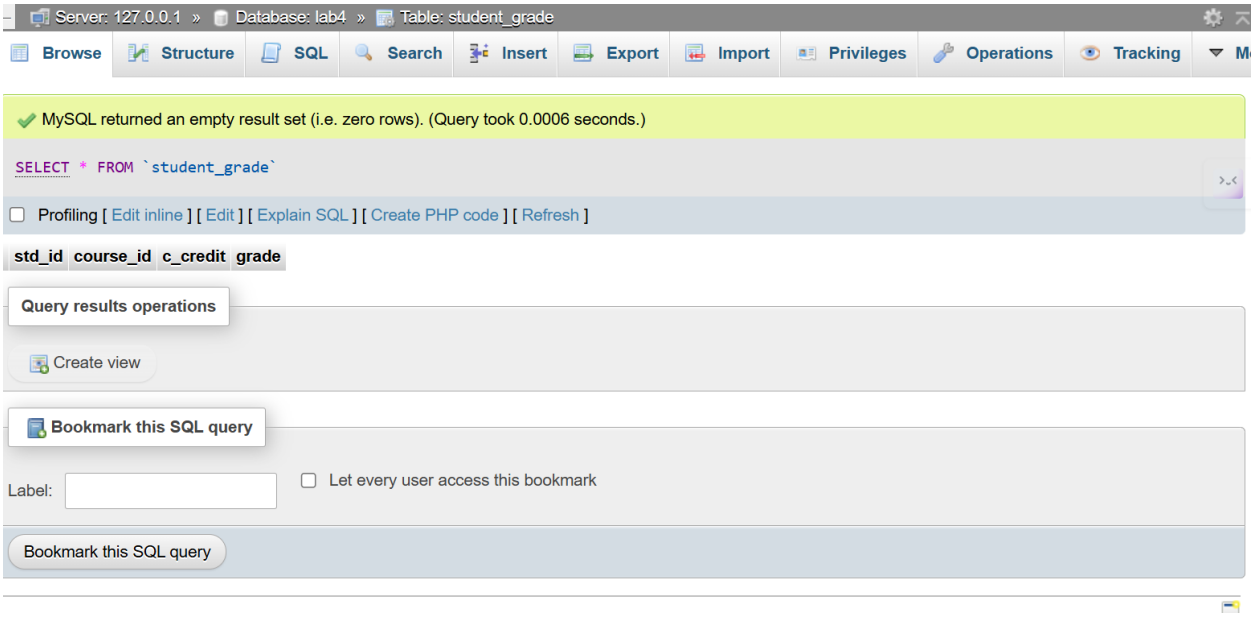
Step-01: SQL Queries (Create Table)



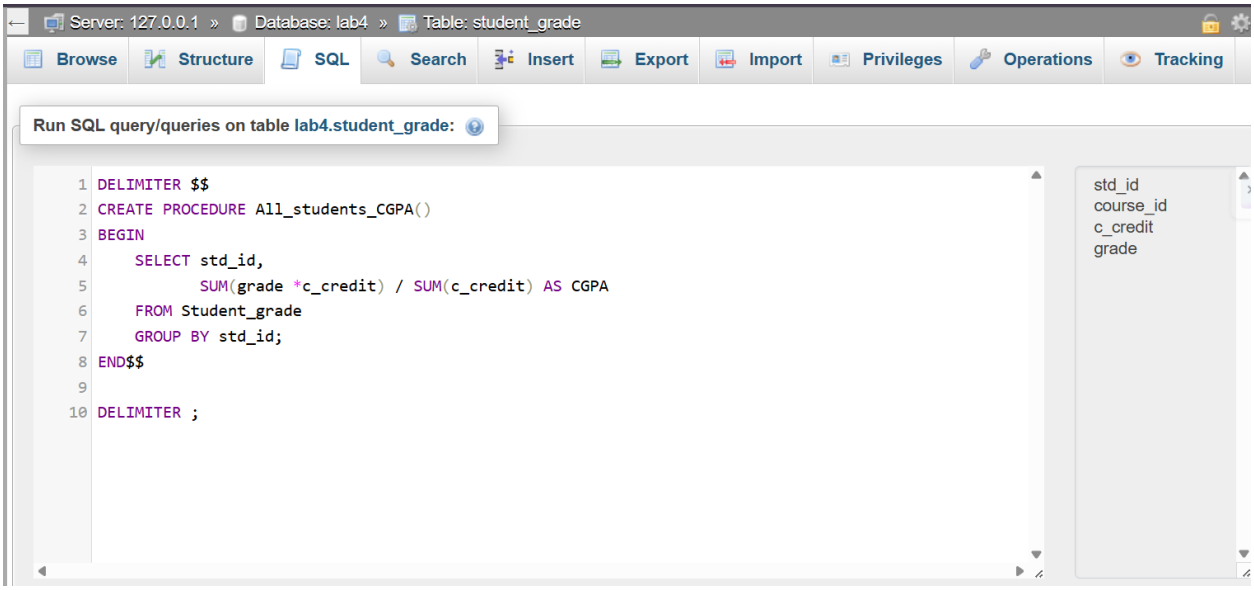
Step-02: Success of Compilation



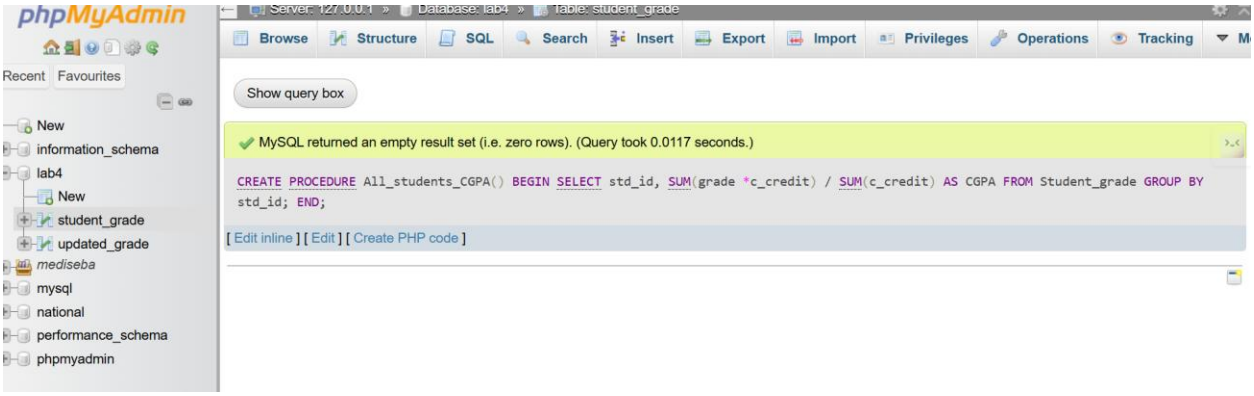
Step-03: Output



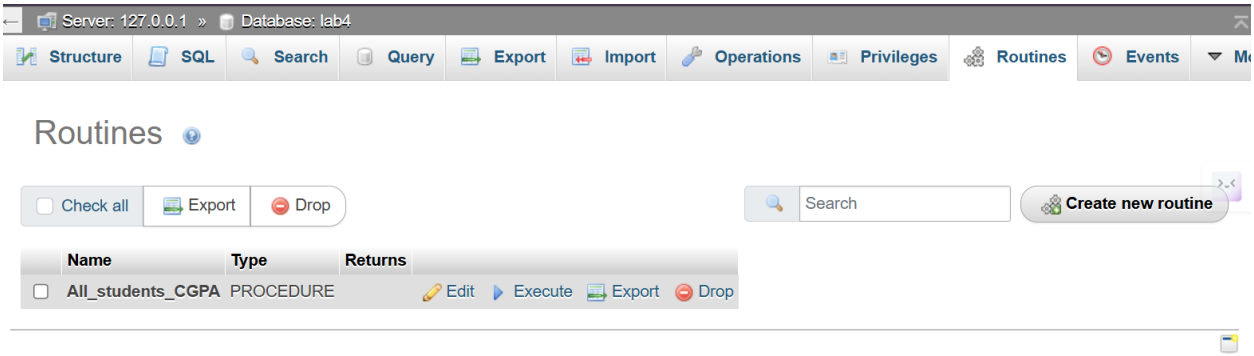
Step-04: SQL Queries (Create the stored procedure)



Step-05: Success of Compilation & Output



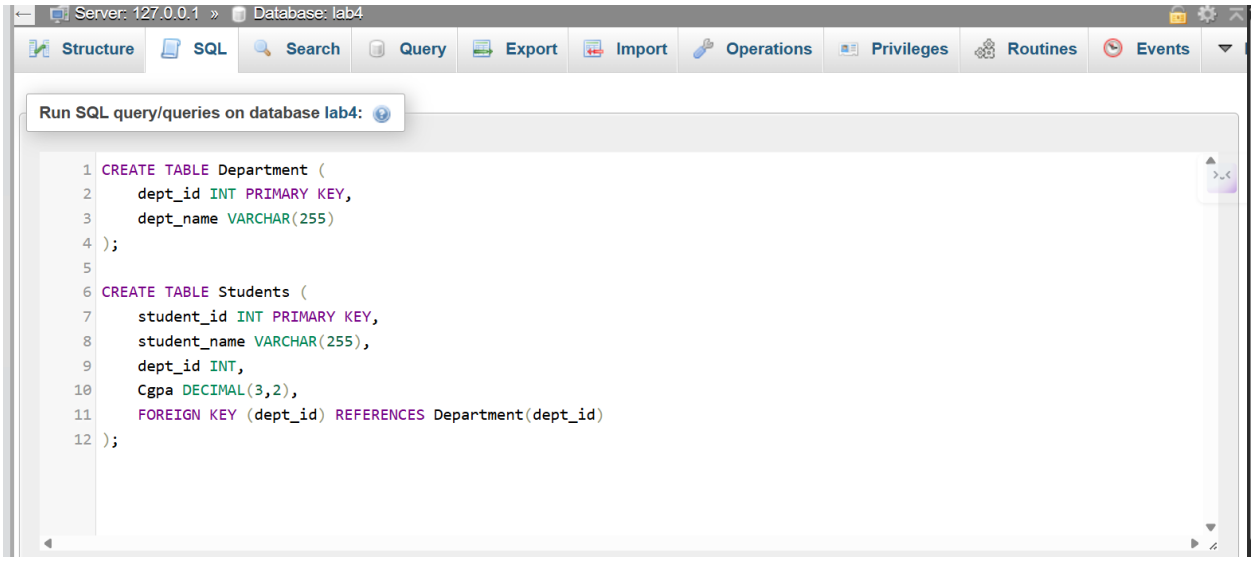
Step-06: Output



Q3. Create a SQL view Avg_Result to display the average CGPA per department, showing:

- **dept_id**
- **dept_name**
- **avg(cgpa)**

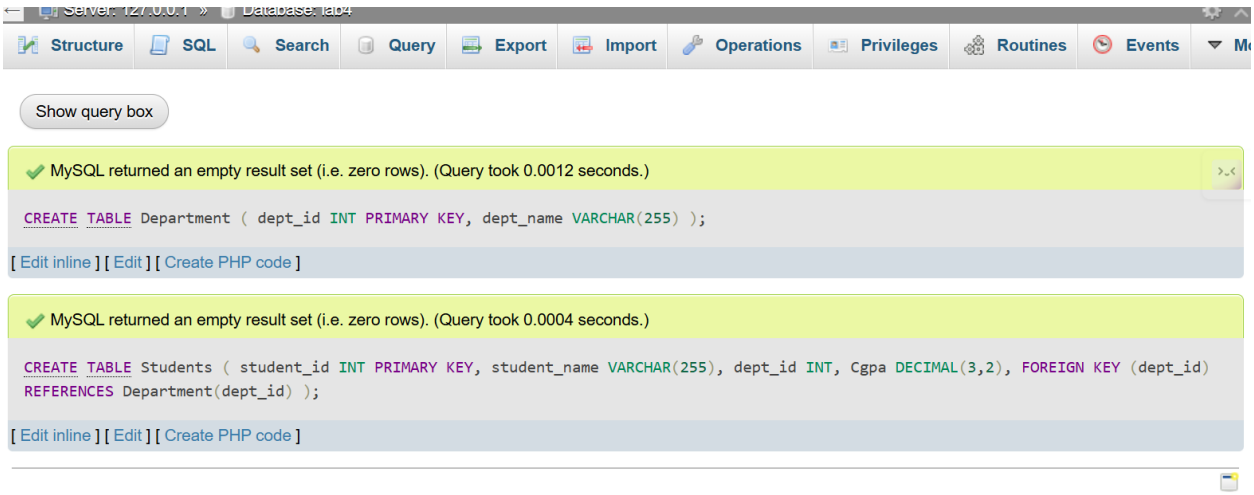
Step-01: SQL Queries (Create Table)



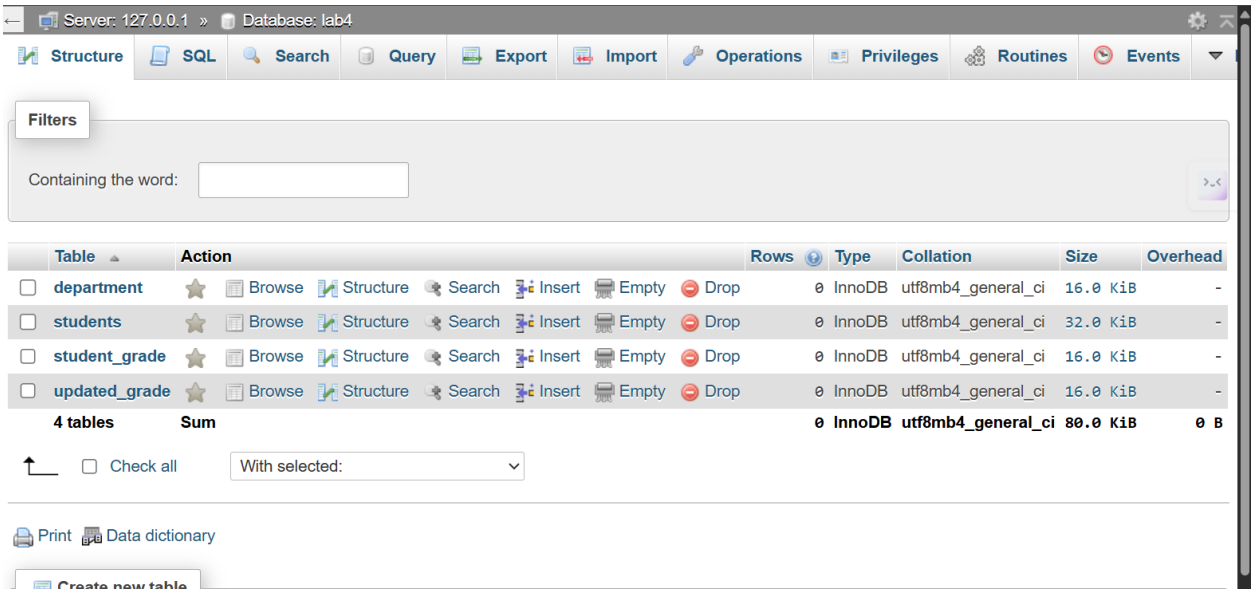

```sql
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(255)
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(255),
    dept_id INT,
    Cgpa DECIMAL(3,2),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

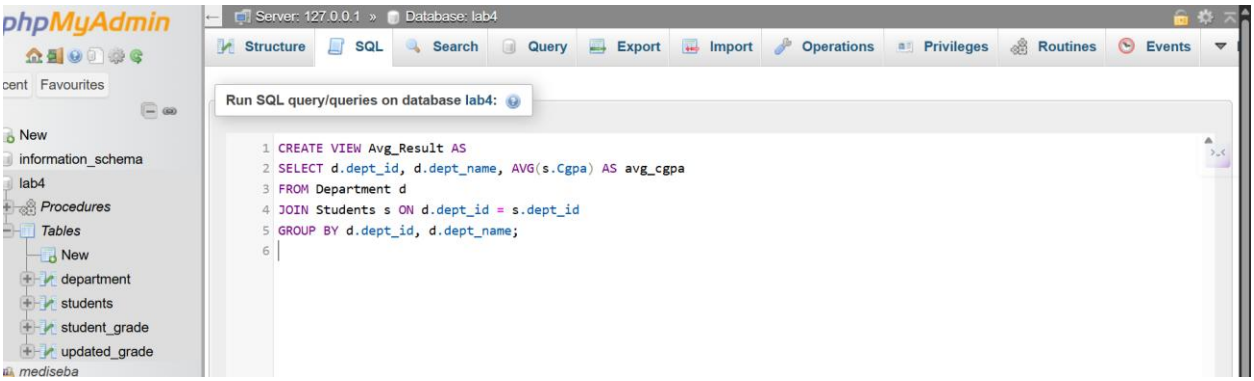
Step-02: Success of Compilation - Table



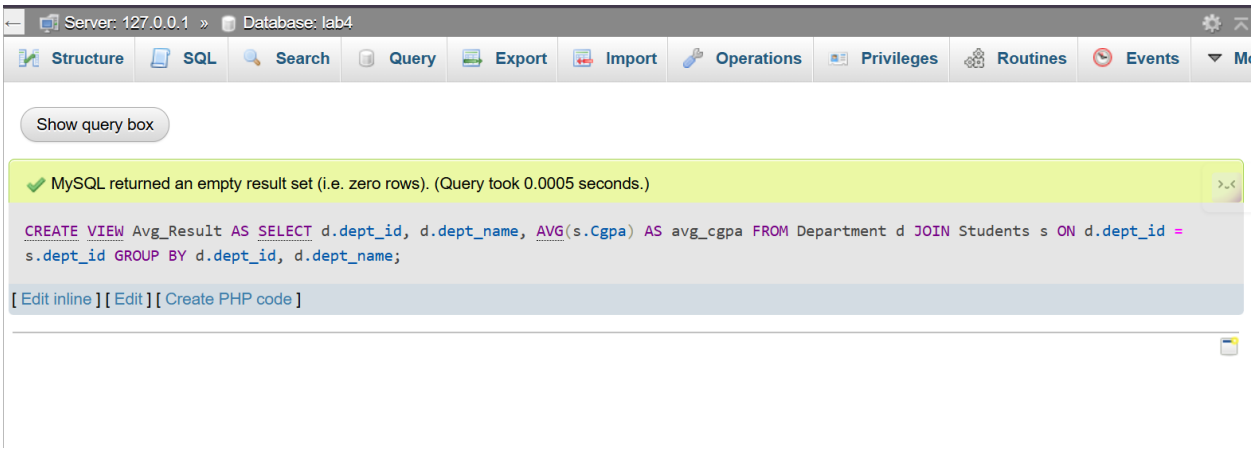
Step-03: Output



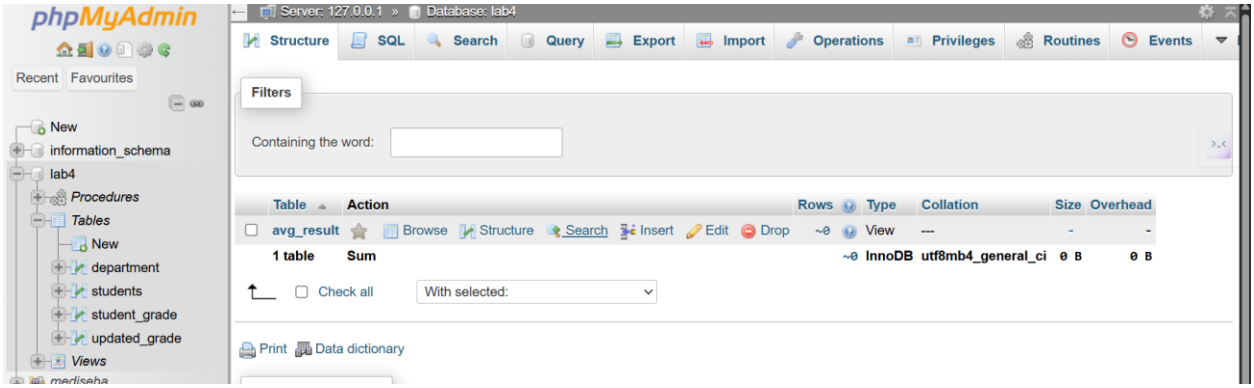
Step-04: SQL Queries (Create the view)



Step-05: Success of Compilation



Step-06: Output



6. Observation:

- Using **OLD** and **NEW** keywords in triggers is essential for capturing historical data accurately.
- For CGPA calculation, ensuring that **c_credit** is not NULL is vital for obtaining mathematically correct results.
- SQL Views help simplify complex queries and improve readability.

7. Appendix:

Appendix A: Course Outcomes, Complex Engineering Problems (EP) and Complex Engineering Activities (EA) Addressing.

Appendix A: Course Outcome (CO) Attainment

SN	COs	Attainment	Justification
01	CO1	Yes	These lab activities enhance understanding of database programming concepts through implementation of triggers, stored procedures, and SQL views.
02	CO2	Yes	Demonstrates automation of database operations and logical implementation of data processing using SQL procedural statements.
03	CO3	Yes	The experiment directly fulfills CO3 by applying advanced SQL programming techniques including triggers, procedures, and views for solving database problems.

Appendix B: Addressing Complex Engineering Problems (EP)

SN	Engineering Problem (EP)	Attainment	Related (CO)	Justification
01	EP1: Depth of Knowledge	Yes	CO3	Requires understanding of advanced SQL concepts and procedural database programming.
02	EP2: Conflicting Requirements	No	CO3	No conflicting requirements involved in this structured academic task.
03	EP4: Familiarity of Issues	Yes	CO3	Covers familiar database automation and reporting problems.
04	EP7: Interdependence	Yes	CO3	Demonstrates dependency between tables and procedural logic in database operations.